

Computer Music Systems

A Brief Genealogy

Anxe Faraldo

(updated: September 2024)

Lejaren Hiller - *Illiac Suite* for String Quartet (1956)

Fourth experiment: *tanto presto che possibile*

Lejaren Hiller: *Illiac Suite*, 4th experiment

For string quartet, 1956

Non-realtime systems: Music-N & Csound

- Max Matthews MUSIC-N languages (1957...)
- Orchestra vs. score metaphor
- UGens...
- Audio vs. control rate

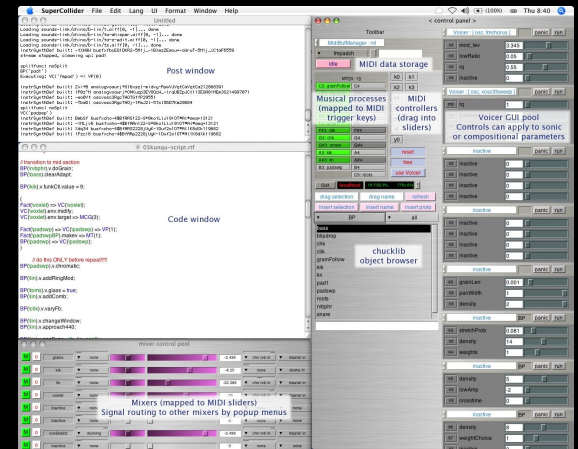
```
<CsoundSynthesizer>
<CsInstruments>

instr 1
aOut vco2 1, p4
out aOut
endin

</CsInstruments>
<CsScore>
i1 0 1 100
i1 1 1 200
i1 2 1 300
</CsScore>
</CsoundSynthesizer>
```

Real-time systems: Supercollider

- By James McCarthy (1996)
- Server/Client interpreted model
- Real-time Synthesis for algorithmic composition and live coding
- Extremely extensible and customisable
- Supports basic GUI development via Qt
- Live-coding & algorave 'standard'

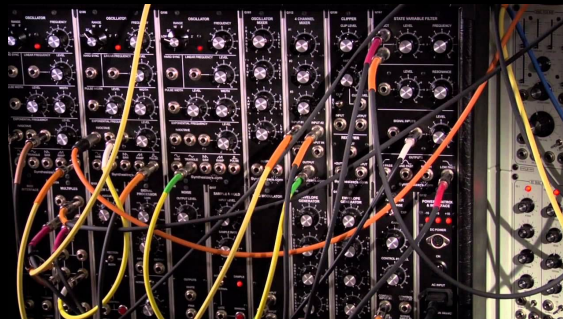


Real-time systems: Chuck

- Ge Wang and Perry Cook (2003)
- Command-line based
- Emphasis on real-time, live coding and algorithmic composition
 - *“Strongly timed and on-the-fly language”*
 - Sample-by-sample processing
 - As real-time as it can get!
- Used by the Princeton Laptop Orchestra

Graphical Programming Languages

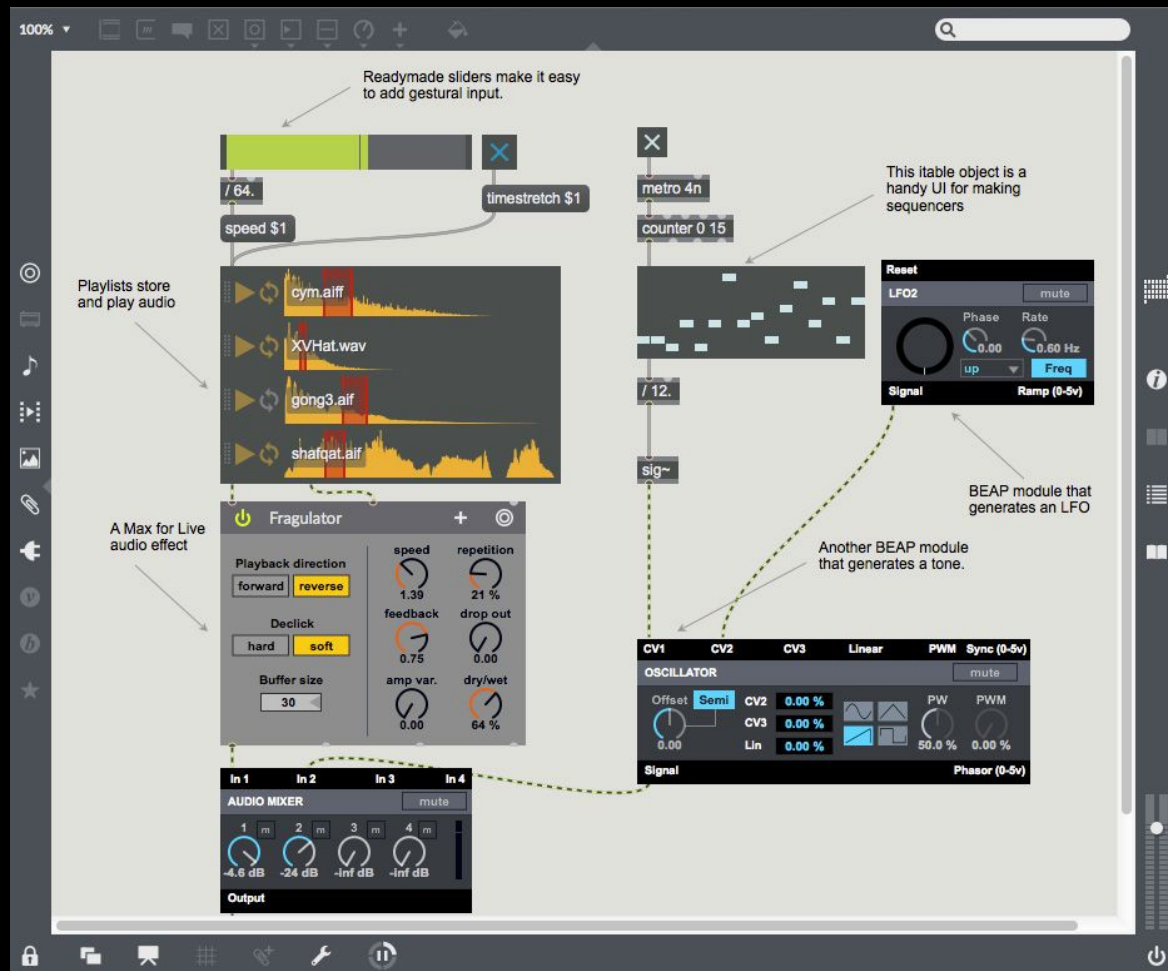
- Inspired by modular synthesizers and circuit design
- Patch cables carry data/audio to/from various modules
- Good and fast prototyping environment
- Realtime and I/O processing
- Edit vs. performance mode: the program is the instrument
- Easy to understand what's going on for non-programmers



Graphical Programming Languages

- Max (Puckette, IRCAM, mid 1980's)
 - Originally data only, controlling external synthesis equipment
- Pure Data [Pd] (Puckette, 1996)
 - Open source
 - Added audio processing, soon to be incorporated in Max
- Max/MSP (Max Signal Processing, 1997)
 - Developed and commercialized by David Zicarelli and Cycling '74
 - Now simply called Max

Max/MSP



Pros & Cons: Max

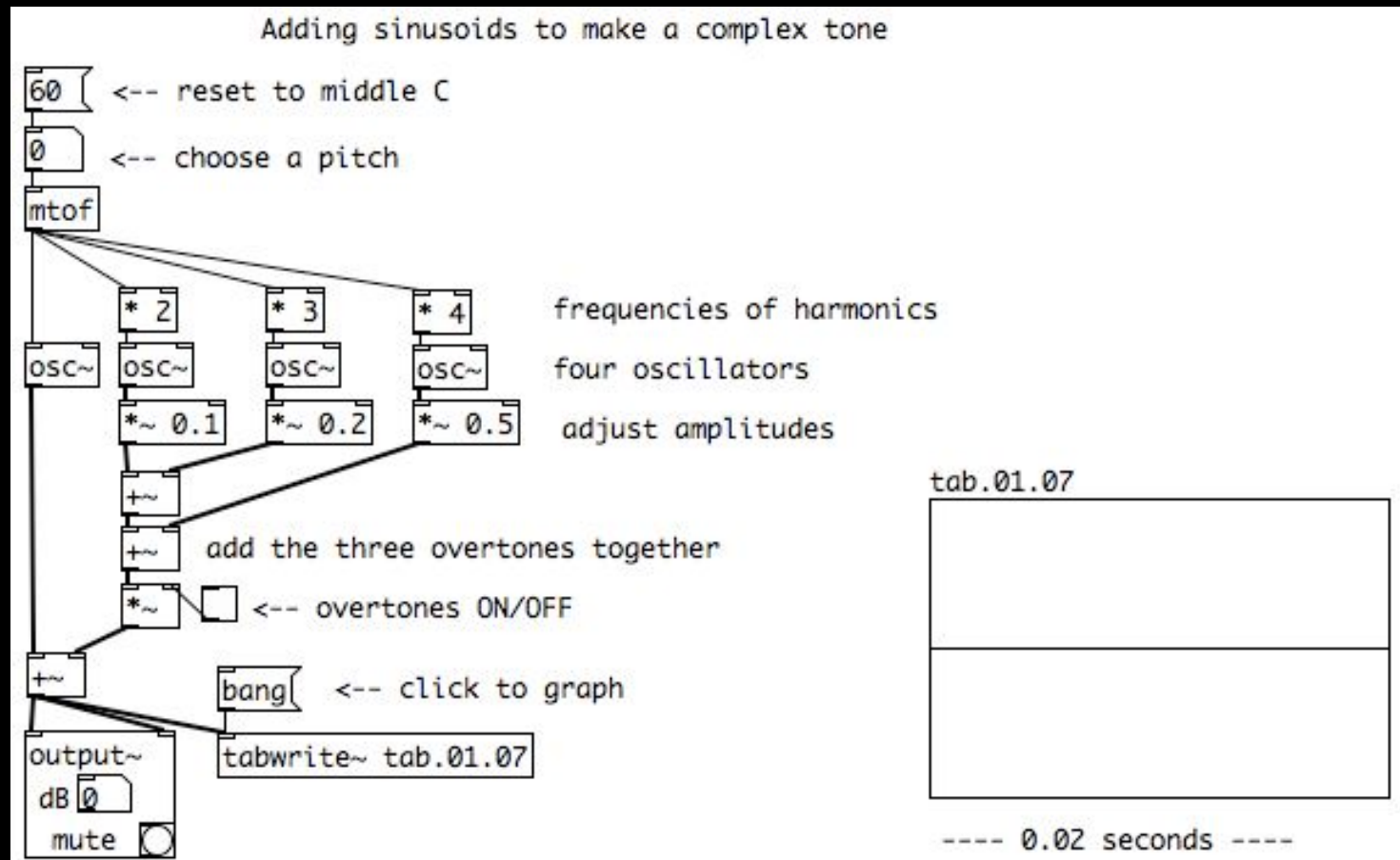
- Pros:

- Mature, well supported with well written documentation
- Many examples and resources (sounds, patches etc.)
- Modern interface
- Ableton Live integration (MaxForLive)

- Cons:

- Resourceful in terms of money and computation power
- Only runs on Mac and Windows

Pure Data



Pros & Cons: Pd

- Pros:

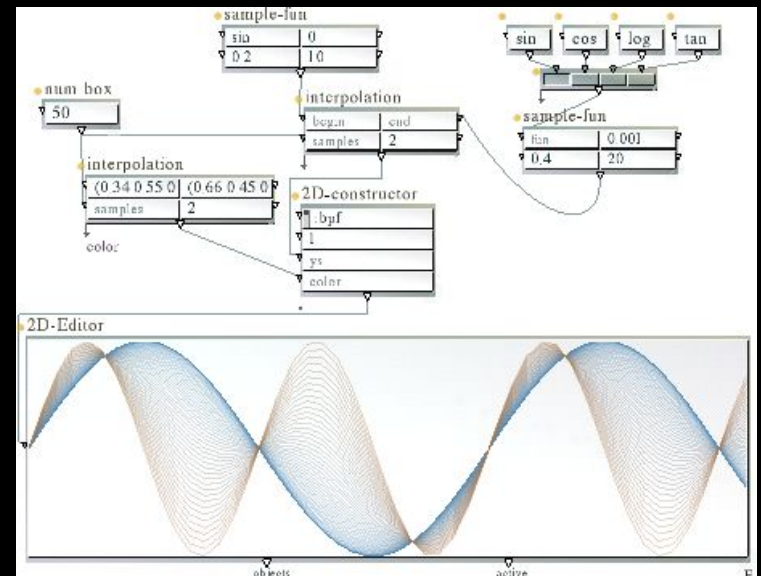
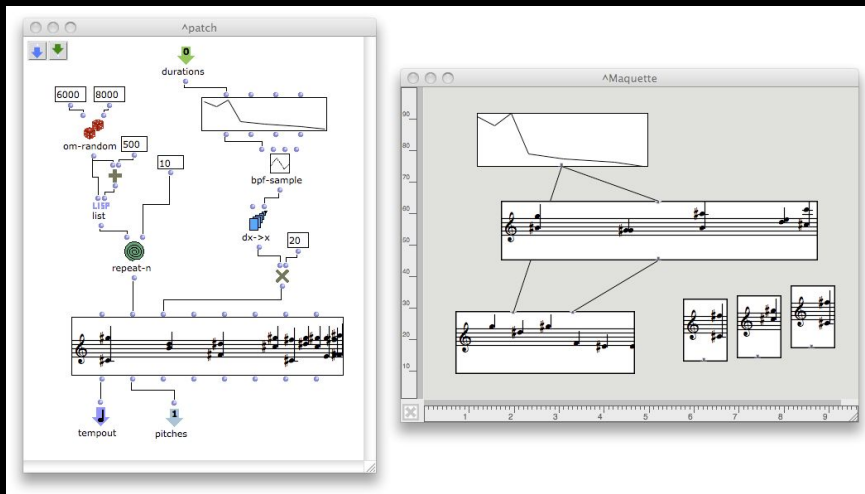
- Open-source and free
- Portable: runs on nearly everything (Mac, Win, Linux, iOS, Android, raspberry pi, web)
- Small language: good for learning!

- Cons:

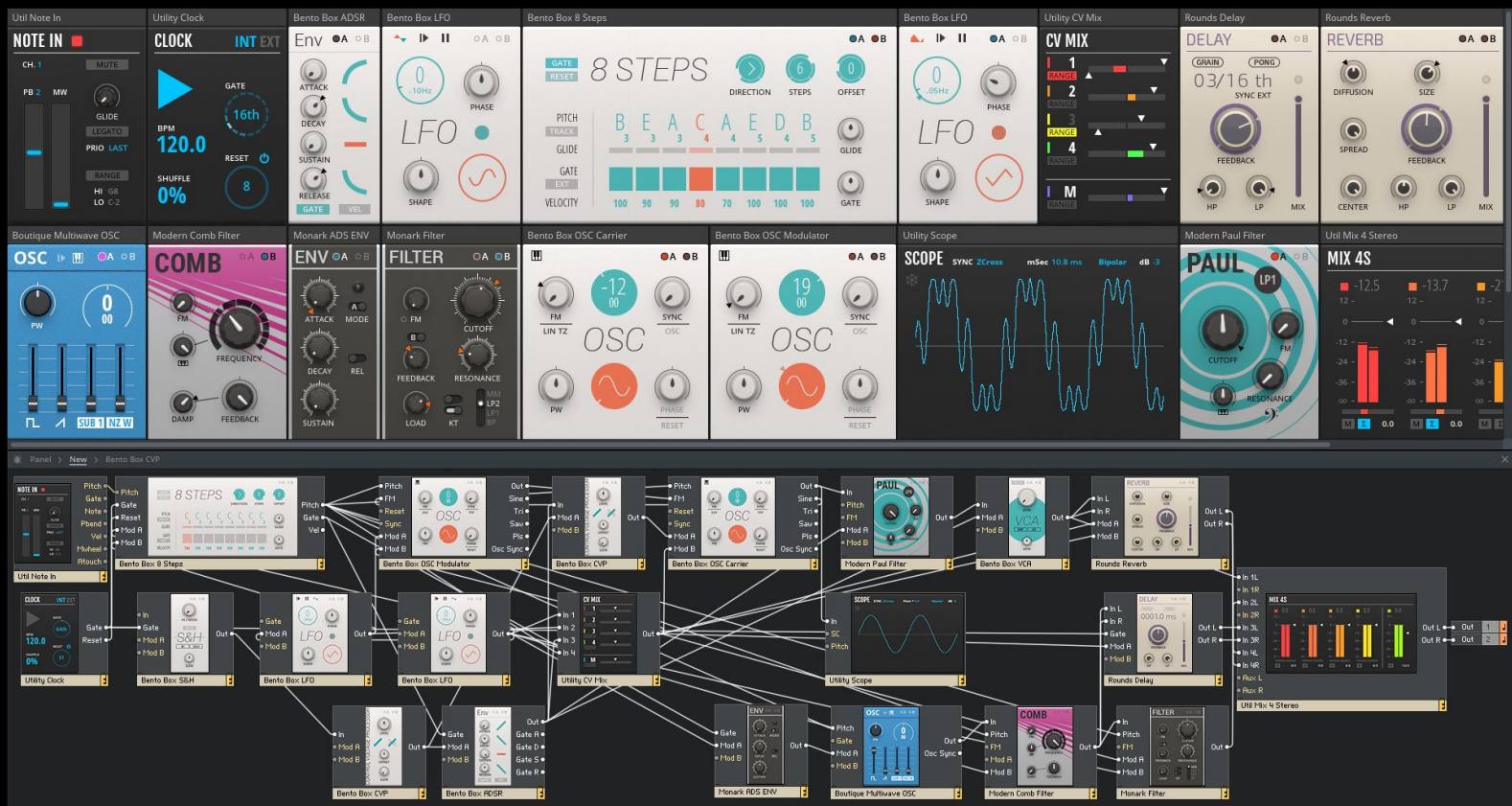
- Documentation can be lacking sometimes
- Community-based support
- Ugly interface for some
- Small language: Some things are hard to do in Pd

Patchwork and Open Music

Concerned with Computer-Assisted Composition

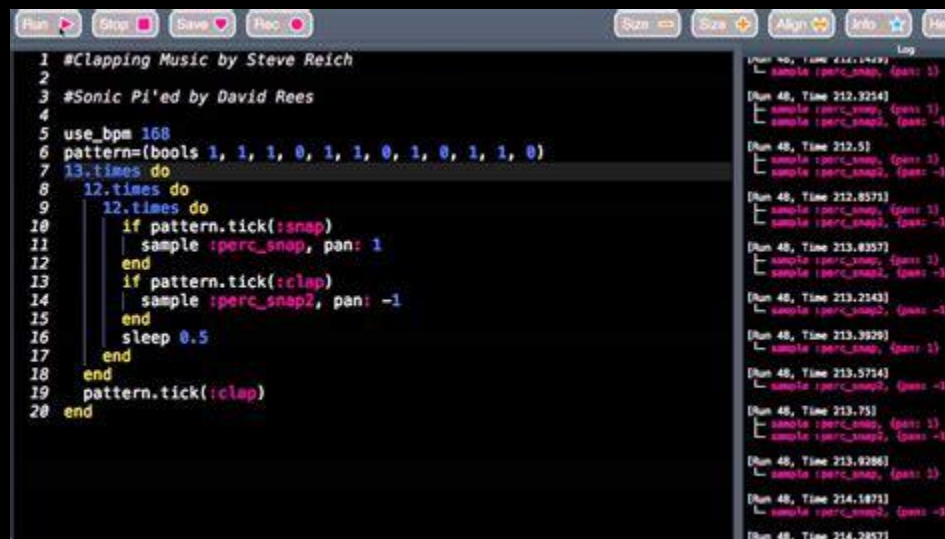


Reaktor (Native Instruments)



Other approaches 1: Live Coding

- A number of languages and extensions are developed with **live coding** in mind (sonic pi, tidal cycles, FoxDot...)
- **SONIC PI** is a paradigmatic example on how to learn to code through music programming



```
1 #Clapping Music by Steve Reich
2
3 #Sonic Pi'ed by David Rees
4
5 use_bpm 168
6 pattern=(bools 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0)
7 13.times do
8   12.times do
9     if pattern.tick(:snap)
10      sample :perc_snap, pan: 1
11    end
12    if pattern.tick(:clap)
13      sample :perc_snap2, pan: -1
14    end
15    sleep 0.5
16  end
17 end
18 pattern.tick(:clap)
19 end
20 end
```

Log

```
[Run 48, Time 212.1429]
└─ sample :perc_snap, (pan: 1)
[Run 48, Time 212.3254]
└─ sample :perc_snap, (pan: 1)
└─ sample :perc_snap2, (pan: -1)
[Run 48, Time 212.5]
└─ sample :perc_snap, (pan: 1)
└─ sample :perc_snap2, (pan: -1)
[Run 48, Time 212.6571]
└─ sample :perc_snap, (pan: 1)
└─ sample :perc_snap2, (pan: -1)
[Run 48, Time 212.8197]
└─ sample :perc_snap, (pan: 1)
└─ sample :perc_snap2, (pan: -1)
[Run 48, Time 213.2143]
└─ sample :perc_snap2, (pan: -1)
[Run 48, Time 213.3929]
└─ sample :perc_snap, (pan: 1)
[Run 48, Time 213.5714]
└─ sample :perc_snap2, (pan: -1)
[Run 48, Time 213.75]
└─ sample :perc_snap, (pan: 1)
└─ sample :perc_snap2, (pan: -1)
[Run 48, Time 213.9286]
└─ sample :perc_snap, (pan: 1)
[Run 48, Time 214.1071]
└─ sample :perc_snap2, (pan: -1)
[Run 48, Time 214.2857]
```

Other approaches 2: Extending DAWs

- Modern DAW's (Ableton, Logic, Reaper, Cubase) include tools for advanced audio and midi processing, virtual instruments and effects that facilitate the workflow.
- Recently, these easily integrate programming extensions for tailored purposes:
 - Max4Live (ableton and max)
 - Camomile / Plugdata (run Pd as AU/VST everywhere!)
 - JS Reaper language...

Other approaches 3: Python



- General purpose language.
- *Lingua Franca* in academic and scientific research
- All sorts of libraries for sound synthesis and analysis, scientific computing, score analysis, musicology, artificial intelligence...
- Interface to other sound synthesis programmes such as supercollider or Pd.
- *PYO, SCAMP, Music21, FoxDot, etc...*

Other approaches 4: Web

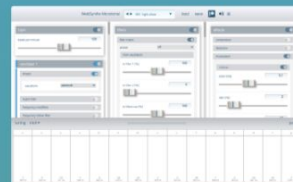
- WEB audio API is a great tool to develop audio applications
- Javascript allows interaction on the web browser



WebSynths
Browser-based musical instruments



WebSynths Grooves
The internet beat box & sample library



WebSynths Microtonal
The deepest internet synthesizer



WebSynths is made possible by support from Patrons. Please consider becoming a Patron for only \$3 a month.



WebSynths on Facebook



Share WebSynths



info@websynths.com

WebSynths.org Learn more about browser-based instruments and the WebAudio API

The End

anxefaraldo@gmail.com